

Kadiatou Sidibe

MLOps

Asynchronous AI Inference System Write Up

1. End-to-End System Description

The system is designed as a decoupled, asynchronous machine learning pipeline designed for breast cancer classification organized into three architectural modules. The orchestration module utilizes Apache Airflow, where a DAG manages the model lifecycle. It handles data ingestion, model training using Scikit-Learn, and serializing the model to an Amazon S3 bucket acting as a Model Registry. It then triggers a producer task to populate Amazon SQS with inference requests. The messaging component (SQS) acts as the "buffer" between data generation and data processing to ensure that inference requests are persisted even if the processing cluster is temporarily unavailable or scaling up. Finally, the inference stage consists of a containerized Python application runs within a Kubernetes cluster. These pods poll SQS, download the latest model from S3, perform the classification, and write the final prediction as a JSON file back to S3 for storage.

2. Why use a queue instead of direct API calls?

A queue provides more durability and better load leveling. In a direct synchronous API setup, the producer must wait for the consumer to finish its task before it can continue. If the consumer is slow or down, the producer fails. For example, when there are traffic spikes such as 2,000 requests arrive at once, an API might reach its connection limit and crash. A queue simply grows in size, allowing the consumers to process messages at a steady pace. Also with a queue, the producer doesn't need to know where the consumer is or if it's currently online, it only needs to know how to talk to the queue.

3. What happens if a consumer crashes mid-processing?

This system utilizes the SQS Visibility Timeout process to ensure at least once delivery. When a Kubernetes pod retrieves a message, SQS marks it as invisible to others. If the pod crashes mid-processing, it fails to send the DeleteMessage command. After the timeout period expires, SQS makes the message visible again. A healthy pod will then pick up the message and retry the inference. This ensures that a single pod failure does not result in lost data.

4. Where are the bottlenecks in your system?

The bottlenecks in my system include model loading latency where each pod must download the model from S3. If the model file grows to several gigabytes, the cold start time for new pods increases significantly. Network throughput is another bottleneck where when there is a high traffic, the single SQS queue can become a bottleneck if the number of consumers polling it exceeds the allowed request rate. Additionally, S3 I/O overhead is a factor where writing

thousands of individual small JSON files to S3 is much slower than writing one large batch file due to the overhead of HTTP requests for every single record.

5. One improvement for production

One improvement for a production would be the implementation of a Dead Letter Queue (DLQ). Currently, if a specific inference request contains corrupted data that causes the consumer pod to crash, SQS will keep returning that "poison pill" message to the queue for retry after every visibility timeout. This can lead to an infinite loop of crashes and wasted compute resources. By configuring a DLQ with a redrive policy, the system would automatically move any message that fails processing more than three times into a separate, isolated queue. This allows the main pipeline to continue running smoothly while alerting engineers to manually inspect and debug the problematic data in the DLQ, ensuring higher system reliability.